

ISTQB® CTFL v4.0

Chapter 5

Managing the Test Activities



[Syllabus-numbered](#) • [interactive](#) • [speaker notes](#)

Chapter 5 learning route

Syllabus numbering retained: 5.1 → 5.5



5

5.1

Test planning

5.2

Risk management

5.3

Monitoring, control,
completion

5.4

Configuration management

5.5

Defect management



INTERACTIVE: Ask learners which management activity is most visible in their organization.

Chapter 5 learning route

Syllabus numbering retained: 5.1 → 5.5



5

5.1 Test Planning

- FL-5.1.1 (K2) Exemplify the purpose and content of a test plan
- FL-5.1.2 (K1) Recognize how a tester adds value to iteration and release planning
- FL-5.1.3 (K2) Compare and contrast entry criteria and exit criteria
- FL-5.1.4 (K3) Use estimation techniques to calculate the required test effort
- FL-5.1.5 (K3) Apply test case prioritization
- FL-5.1.6 (K1) Recall the concepts of the test pyramid
- FL-5.1.7 (K2) Summarize the testing quadrants and their relationships with test levels and test types

5.2 Risk Management

- FL-5.2.1 (K1) Identify risk level by using risk likelihood and risk impact
- FL-5.2.2 (K2) Distinguish between project risks and product risks
- FL-5.2.3 (K2) Explain how product risk analysis may influence thoroughness and scope of testing
- FL-5.2.4 (K2) Explain what measures can be taken in response to analyzed product risks

5.3 Test Monitoring, Test Control and Test Completion

- FL-5.3.1 (K1) Recall metrics used for testing
- FL-5.3.2 (K2) Summarize the purposes, content, and audiences for test reports
- FL-5.3.3 (K2) Exemplify how to communicate the status of testing

5.4 Configuration Management

- FL-5.4.1 (K2) Summarize how configuration management supports testing

5.5 Defect Management

- FL-5.5.1 (K3) Prepare a defect report



INTERACTIVE: Ask learners which management activity is most visible in their organization.

5.1 Test Planning

5.1.1 Purpose and content of a test plan

A test plan describes objectives, resources and processes for a test project.



Purpose

- Documents the means and schedule for achieving test objectives
- Helps to ensure that the performed test activities will meet the established criteria
- Serves as a means of communication with team members and other stakeholders
- Demonstrates that testing will adhere to the existing test policy and test strategy (or explains why the testing will deviate from them)



Thinking tool

- Forces testers to confront risks, schedules, people, tools, costs and effort
- Helps anticipate future challenges



Typical content

- Context of testing (e.g., scope, test objectives, constraints, test basis)
- Assumptions and constraints of the test project
- Stakeholders (e.g., roles, responsibilities, relevance to testing, hiring and training needs)
- Communication (e.g., forms and frequency of communication, documentation templates)
- Risk register (e.g., product risks, project risks)
- Test approach (e.g., test levels, test types, test techniques, test deliverables, entry criteria and exit criteria, independence of testing, metrics to be collected, test data requirements, test environment requirements, deviations from the organizational test policy and test strategy)
- Budget and schedule



INTERACTIVE: Ask learners: which test plan section prevents the most misunderstanding?



5.1.2 Tester contribution to release and iteration planning

Testers add value before test execution starts.

In iterative SDLCs, typically two kinds of planning occur:



Release planning

- **release of a product**, defines and re-defines the product backlog, and may involve refining larger user stories into a set of smaller user stories.
- serves as the **basis for the test approach and test plan** across all iterations. Testers involved in release planning participate in writing testable user stories and acceptance criteria (see section 4.5), participate in project and quality risk analyses (see section 5.2), estimate test effort associated with user stories (see section 5.1.4), determine the test approach, and plan the testing for the release.



Iteration planning

- To the end of a single iteration and is concerned with the iteration backlog. Testers involved in iteration planning participate in the detailed risk analysis of user stories, determine the testability of user stories, break down user stories into tasks (particularly testing tasks), estimate test effort for all testing tasks, and identify and refine functional and non-functional aspects of the test object.



INTERACTIVE: Ask learners how testers can improve a backlog before coding starts.

5.1.3 Entry criteria and exit criteria

Entry asks “can we start?”; exit asks “can we stop?”



Entry criteria

- Preconditions for undertaking a given test activity. If entry criteria are not met, it is likely that the activity will prove more difficult, more time-consuming, more costly, and more risky.
 - Examples: availability of resources (e.g., people, tools, environments, test data, budget, time), availability of testware (e.g., test basis, testable requirements, user stories, test cases), and initial quality level of a test object (e.g., all smoke tests have passed).
- Agile term: Definition of Ready



Exit criteria

- What must be achieved to declare completion
 - Examples: measures of thoroughness (e.g., achieved level of coverage, number of unresolved defects, defect density, number of failed test cases), and completion criteria (e.g., planned tests have been executed, static testing has been performed, all defects found are reported, all regression tests are automated).
- Agile term: Definition of Done

- Entry and exit criteria should be defined for each test level and test type, and will differ based on the test objectives.



INTERACTIVE: Ask learners to write one entry and one exit criterion for system testing.

5.1.4 Estimation techniques

Estimates predict test-related work and always contain assumptions.

Test effort estimation involves predicting the amount of test-related work needed to meet the objectives of a test project.

It is important to make it clear to the stakeholders that the **estimate is based on a number of assumptions** and is always subject to estimation error.

Estimation for **small tasks is usually more accurate than for the large ones**. Therefore, when estimating a large task, it can be decomposed into a set of smaller tasks which then in turn can be estimated.



Estimation based on Ratios

- Use historical organizational ratios
- Example: development-to-test effort ratio
- Best when similar past data exists



Extrapolation

- Use early current-project measurements
- Project future effort from recent trend
- Useful in iterative SDLCs



Expert-based

- **Wideband Delphi / Planning Poker**
- **Three-point estimation**
- Useful when historical data is weak



INTERACTIVE: Ask: which technique would you use for a new project with no internal history?

5.1.4 Estimation techniques

Estimates predict test-related work and always contain assumptions.

5



Estimation based on Ratios

In this metrics-based technique, **figures are collected from previous projects** within the organization, which makes it possible to derive “standard” ratios for similar projects. The ratios of an organization’s own projects (**e.g., taken from historical data**) are generally the best source to use in the estimation process. These standard ratios can then be used to estimate the test effort for the new project.

For example, if in the previous project the development-to-test effort ratio was 3:2, and in the current project the development effort is expected to be 600 person-days, the test effort can be estimated to be 400 person-days.



INTERACTIVE: Ask: which technique would you use for a new project with no internal history?

5.1.4 Estimation techniques

Estimates predict test-related work and always contain assumptions.

5



Extrapolation

In this metrics-based technique, **measurements are made as early as possible** in the current project to gather the data. Having enough observations, the effort required for the remaining work can be approximated by extrapolating this data (usually by applying a mathematical model). This method is very suitable in iterative SDLCs.

For example, a team can extrapolate the testing effort in the fourth iteration as an average of the effort in the last three iterations. If the effort in the last three iterations was 30, 32, and 25 person-days, respectively, the extrapolation of effort for the fourth iteration would be $(30+32+25) / 3 = 29$ person-days.

****Extrapolation: the action of estimating or concluding something by assuming that existing trends will continue or a current method will remain applicable**



INTERACTIVE: Ask: which technique would you use for a new project with no internal history?

5.1.4 Estimation techniques

Estimates predict test-related work and always contain assumptions.

5



Expert-based (Wideband Delphi)

- In this iterative, expert-based technique, experts make experience-based estimations.
- Each expert, in isolation, estimates the effort. The results are collected and if there are deviations that are out of range of the agreed upon boundaries, the experts discuss their current estimates. Each expert is then asked to make a new estimation based on that feedback, again in isolation. This process is repeated until a consensus is reached.
- **Planning Poker** is a variant of Wideband Delphi, commonly used in Agile software development. In Planning Poker, estimates are usually made using cards with numbers that represent the effort size.



INTERACTIVE: Ask: which technique would you use for a new project with no internal history?

5.1.4 Estimation techniques

Estimates predict test-related work and always contain assumptions.

5



Planning poker cards most often contain values derived from the so-called Fibonacci sequence, although for larger values, there may be some deviations.

In the Fibonacci sequence, each successive term is the sum of the previous two. The following values are most often used:

1, 2, 3, 5, 8, 13, 20, 40, 100

The last two values are no longer the sum of the previous two. This is because they are large enough that rounded values are assumed here

- If a team estimates, for example, the effort required to implement and test some user story and most experts throw a 40 or 100 card on the table, this means that the story is large enough that it is probably an epic and should be broken down into several smaller stories, each of which can already be reasonably estimated.
- If a card with “?” is thrown, it means that the expert does not have enough information to do the estimation.
- A value of “0,” on the other hand, means that the expert considers the story to be trivial to implement and that the time spent on it will be negligible.
- A card with coffee picture means “I am tired, let us do the break and grab some coffee!”.



INTERACTIVE: Ask: which technique would you use for a new project with no internal history?

5.1.4 Estimation techniques

Estimates predict test-related work and always contain assumptions.



Three-point estimation - Use optimistic, most likely and pessimistic estimates.

In this metrics-based technique, **figures are collected from previous projects** within the organization, which makes it possible to derive “standard” ratios for similar projects. The ratios of an organization’s own projects (**e.g., taken from historical data**) are generally the best source to use in the estimation process. These standard ratios can then be used to estimate the test effort for the new project.

Three estimations are made by the experts:

- the most optimistic estimation (a)
- the most likely estimation (m)
- the most pessimistic estimation (b).

For example, if in the previous project the development-to-test effort ratio was 3:2, and in the current project the development effort is expected to be 600 person-days, the test effort can be estimated to be 400 person-days.



Formula

- $E = (a + 4m + b) / 6$
- $SD = (b - a) / 6$
- Report estimate as $E \pm SD$ when useful



Example

- $a = 6, m = 9, b = 18$
- $E = 10$
- $SD = 2$
- Estimated effort = 10 ± 2 person-hours



INTERACTIVE: Calculate E and SD for $a=4, m=7, b=16$.

5.1.5 Test case prioritization

Prioritization determines the execution order of test cases.



Once the test cases and test procedures are specified and assembled into test suites, these test suites **can be arranged in a test execution schedule** that defines the order in which they are to be run. When prioritizing test cases, different factors can be taken into account. The most commonly used test case prioritization strategies are as follows:



Risk-based

- Run tests covering most important risks first
- Useful when time is constrained



Coverage-based

- Run tests with highest coverage first
- Additional coverage prioritization selects the next test with highest new coverage



Requirements-based

- Run tests linked to highest-priority requirements first
- Priorities come from stakeholders



INTERACTIVE: Given limited time, which tests should run first: high-risk or high-coverage? Discuss.

5.1.5 Test case prioritization

Prioritization determines the execution order of test cases.

- Ideally, test cases would be ordered to run based on their priority levels, using, for example, one of the above-mentioned prioritization strategies.
- However, this practice may not work if the test cases or the features being tested **have dependencies**. If a test case with a higher priority is dependent on a test case with a lower priority, the lower priority test case must be executed first.
- The order of test execution must also take into account the availability of resources. For example, the required test tools, test environments or people that may only be available for a specific time window.

Fig. 5.8 CFG of the system under test

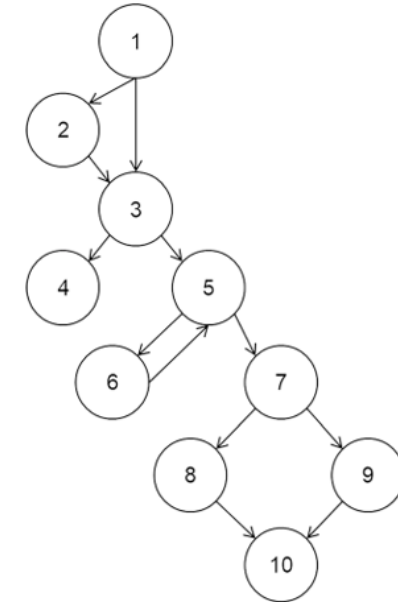


Table 5.1 Existing test cases and the coverage they achieve

Test case	Exercised path	Coverage	Priority
TC1	1→3→5→7→8→10	60%	2
TC2	1→3→5→7→9→10	60%	2
TC3	1→2→3→4	40%	3
TC4	1→3→5→6→5→7→8→10	70%	1



INTERACTIVE: Given limited time, which tests should run first: high-risk or high-coverage? Discuss.



5.1.6 Test pyramid

The pyramid supports test automation and effort allocation.

- The test pyramid is a model showing that different tests may have different **granularity**.
- The test pyramid model supports the team in test automation and in test effort allocation by showing that different goals are supported by different levels of test automation.
- The pyramid layers represent groups of tests:
 - The higher the layer, the lower the test granularity, test isolation and test execution time.
 - Tests in the bottom layer are small, isolated, fast, and check a small piece of functionality, so usually a lot of them are needed to achieve a reasonable coverage.
 - The top layer represents complex, high-level, end-to-end tests. These high-level tests are generally slower than the tests from the lower layers, and they typically check a large piece of functionality, so usually just a few of them are needed to achieve a reasonable coverage.

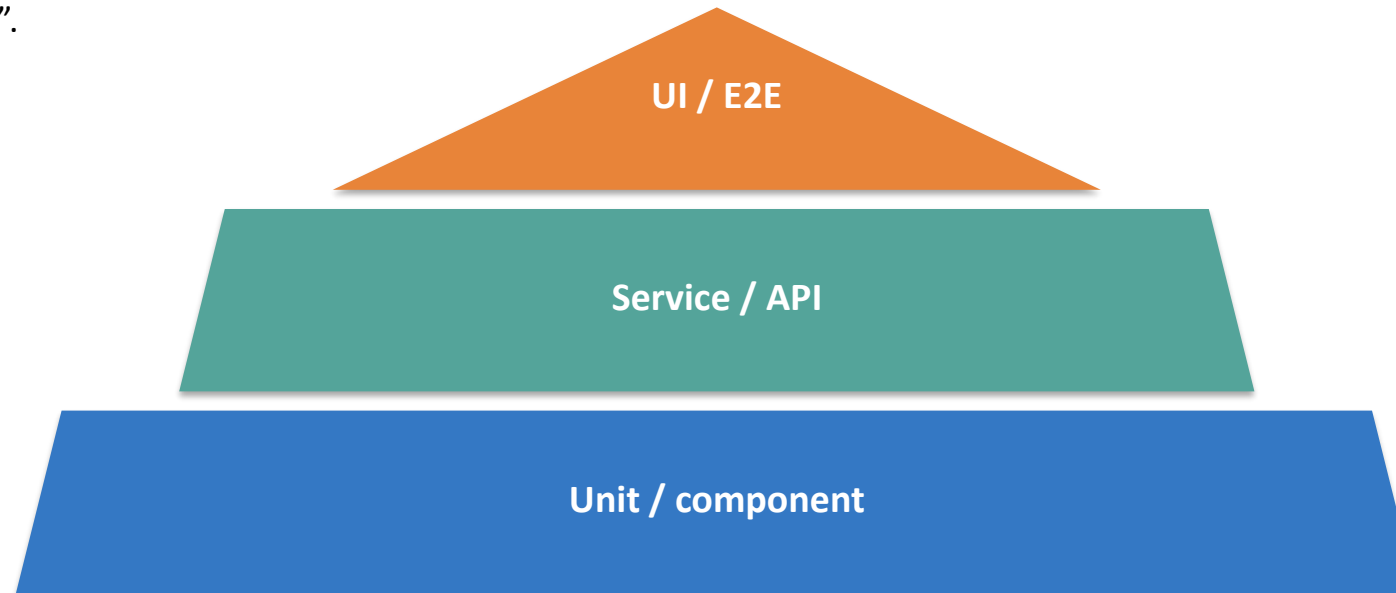


INTERACTIVE: Ask why the pyramid should not be inverted.

5.1.6 Test pyramid

The pyramid supports test automation and effort allocation.

The number and naming of the layers may differ. For example, the original test pyramid model (Cohn 2009) defines three layers: “unit tests”, “service tests” and “UI tests”.



Interpretation

- Lower layers are faster, smaller and more numerous; upper layers are slower, broader and fewer.



INTERACTIVE: Ask why the pyramid should not be inverted.

5.1.7 Testing quadrants

Quadrants help visualize agile testing across business/technology and support/critique dimensions.



The testing quadrants, defined by Brian Marick (Marick 2003, Crispin 2008), **group the test levels with the appropriate test types, activities, test techniques and work products** in the Agile software development.

The model supports test management in **visualizing these to ensure that all appropriate test types and test levels are included in the SDLC** and in understanding that some test types are more relevant to certain test levels than others. This model also provides a way to differentiate and describe the types of tests to all stakeholders, including developers, testers, and business representatives.

In this model, **tests can be business facing or technology facing**. Tests can also support the team (i.e., guide the development) or critique the product (i.e., measure its behavior against the expectations). The combination of these two viewpoints determines the four quadrants:

Q2 Business-facing support

Q3 Business-facing critique

Q1 Technology-facing support

Q4 Technology-facing critique

Support team ↔ Critique product | Business-facing ↔ Technology-facing



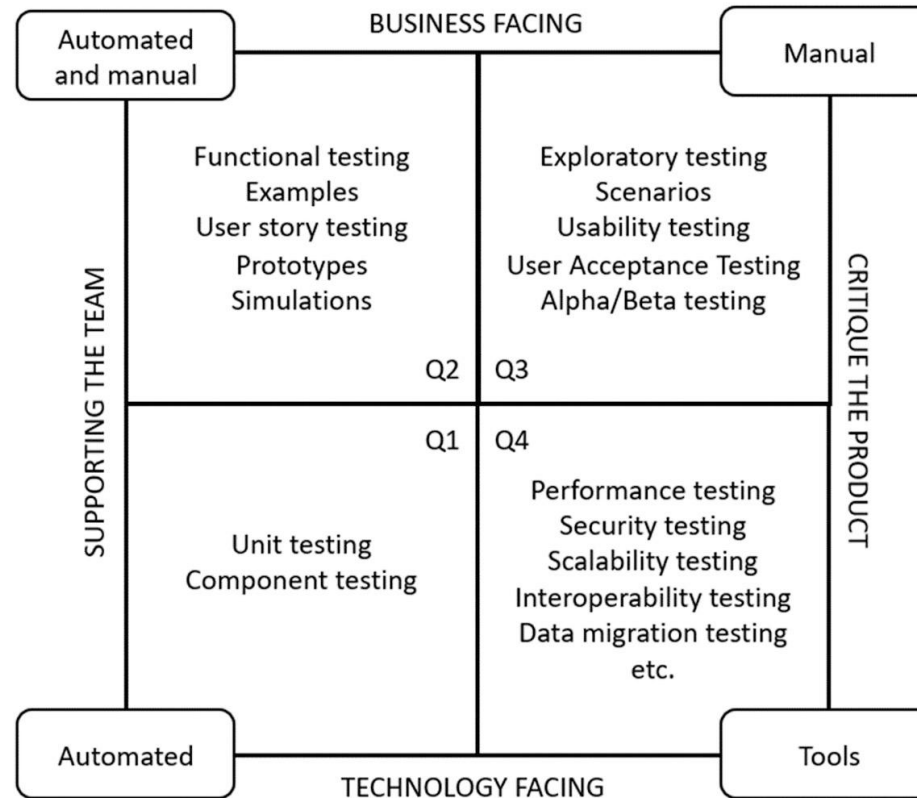
INTERACTIVE: Place performance testing, story tests and UAT into the quadrants.

5.1.7 Testing quadrants

Quadrants help visualize agile testing across business/technology and support/critique dimensions.

Quadrant Q2 (business facing, support the team). This quadrant contains functional tests, examples, user story tests, user experience prototypes, API testing, and simulations. These tests check the acceptance criteria and can be manual or automated.

Quadrant Q1 (technology facing, support the team). This quadrant contains component and component integration tests. These tests should be automated and included in the CI process.



Quadrant Q3 (business facing, critique the product). This quadrant contains exploratory testing, usability testing, user acceptance testing. These tests are user-oriented and often manual.

Quadrant Q4 (technology facing, critique the product). This quadrant contains smoke tests and non-functional tests (except usability tests). These tests are often automated.



INTERACTIVE: Place performance testing, story tests and UAT into the quadrants.

5.2 Risk management

Risk-based testing selects, prioritizes and manages testing based on risk.



Organizations face many internal and external factors that make it uncertain whether and when they will achieve their objectives (ISO 31000). Risk management allows the organizations to increase the likelihood of achieving objectives, improve the quality of their products and increase the stakeholders' confidence and trust.

The main risk management activities are:

- Risk analysis (consisting of risk identification and risk assessment; see section 5.2.3)
- Risk control (consisting of risk mitigation and risk monitoring; see section 5.2.4)

The test approach, in which test activities are selected, prioritized, and managed based on risk analysis and risk control, is called risk-based testing.



Risk analysis

- Risk identification
- Risk assessment
- Builds awareness of product risks



Risk control

- Risk mitigation
- Risk monitoring
- Checks whether actions are effective



Risk-based testing

- Uses risk analysis and risk control to guide scope, priorities and effort



INTERACTIVE: Ask: why does high-risk functionality usually deserve earlier testing?

5.2.1 Risk definition and attributes

Risk level is driven by likelihood and impact.

5



Risk is a potential event, hazard, threat, or situation whose occurrence causes an adverse effect. A risk can be characterized by two factors:



Risk Likelihood

- Probability that the risk occurs
- Greater than zero and less than one
- Can be judged qualitatively or quantitatively



Risk Impact

- Consequence or harm if the risk occurs
- Can include financial, safety, reputation or legal harm
- Higher risk level deserves stronger treatment

These two factors express the risk level, which is a measure for the risk. The higher the risk level, the more important is its treatment.

$\text{Risk level} = \text{Risk likelihood} \times \text{Risk impact}$



INTERACTIVE: Plot examples on a likelihood-impact matrix.

5.2.2 Project risks and product risks

Project risks threaten project success; product risks threaten product quality.



Project risks

Project risks are related to the management and control of the project. Project risks include:

- Organizational issues (e.g., delays in work products deliveries, inaccurate estimates, cost-cutting)
- People issues (e.g., insufficient skills, conflicts, communication problems, shortage of staff)
- Technical issues (e.g., scope creep, poor tool support)
- Supplier issues (e.g., third-party delivery failure, bankruptcy of the supporting company)

mpact schedule, budget or scope



Product risks

Product risks are related to the product quality characteristics (e.g., described in the ISO 25010 quality model).

Examples of product risks include: missing or wrong functionality, incorrect calculations, runtime errors, poor architecture, inefficient algorithms, inadequate response time, poor user experience, security vulnerabilities.

Product risks, when they occur, may result in various negative consequences, including:

- User dissatisfaction
- Loss of revenue, trust, reputation
- Damage to third parties
- High maintenance costs, overload of the helpdesk
- Criminal penalties

May cause injury or death in extreme cases



INTERACTIVE: Classify: “test environment delayed” vs “incorrect interest calculation.”

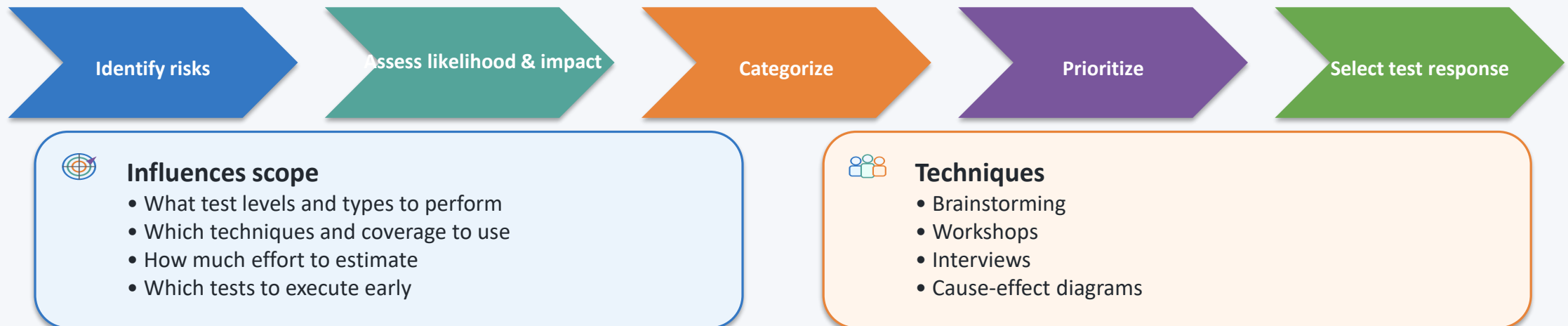
5.2.3 Product risk analysis

Use risk analysis to focus testing where it reduces residual risk most.

From a testing perspective, the goal of product risk analysis is to provide **an awareness of product risk in order to focus the testing effort** in a way that minimizes the residual level of product risk. Ideally, product risk analysis begins early in the SDLC.

Product risk analysis consists of risk identification and risk assessment:

- Risk identification is about **generating a comprehensive list of risks**. Stakeholders can identify risks by using various techniques and tools, e.g., brainstorming, workshops, interviews, or cause-effect diagrams.
- Risk assessment involves: **categorization of identified risks, determining their risk likelihood, risk impact and level, prioritizing, and proposing ways to handle them**. Categorization helps in assigning mitigation actions, because usually risks falling into the same category can be mitigated using a similar approach.
- Risk assessment can use a **quantitative or qualitative approach**, or a mix of them. In the quantitative approach the risk level is calculated as the multiplication of risk likelihood and risk impact. In the qualitative approach the risk level can be determined using a risk matrix.



INTERACTIVE: Ask learners to identify product risks for an online payment module.

5.2.3 Product risk analysis

Use risk analysis to focus testing where it reduces residual risk most.



Product risk analysis may **influence the thoroughness and scope of testing**. Its results are used to:

- Determine the scope of testing to be carried out
- Determine the particular test levels and propose test types to be performed
- Determine the test techniques to be employed and the coverage to be achieved
- Estimate the test effort required for each task
- Prioritize testing in an attempt to find the critical defects as early as possible
- Determine whether any activities in addition to testing could be employed to reduce risk

Example of Risk template and Risk matrix:

Product risk	Likelihood	Impact	Risk priority number	Mitigation
<i>Risk category 1</i>				
<i>Risk 1</i>				
<i>Risk 2</i>				
<i>Risk n</i>				

Example of Risk Template

RISK LEVEL		RISK LIKELIHOOD			
		low	medium	high	very high
RISK IMPACT	low	very low	low	medium	medium
	medium	low	medium	high	high
	high	medium	high	very high	extreme

Example of Risk Matrix

5.2.4 Product risk control

Risk control responds to analyzed product risks.

Product risk control comprises all measures that are taken in response to identified and assessed product risks. Product risk control consists of risk mitigation and risk monitoring.

- **Risk mitigation** involves implementing the actions proposed in risk assessment to reduce the risk level.
- **Risk monitoring** is to ensure that the mitigation actions are effective, to obtain further information to improve risk assessment, and to identify emerging risks.

With respect to **product risk control**, once a risk has been analyzed, several response options to risk are possible, e.g., risk mitigation by testing, risk acceptance, risk transfer, or contingency plan (Veenendaal 2012). Actions that can be taken to mitigate the product risks by testing are as follows:

- Select the testers with the right level of experience and skills, suitable for a given risk type
- Apply an appropriate level of independence of testing
- Conduct reviews and perform static analysis
- Apply the appropriate test techniques and coverage levels
- Apply the appropriate test types addressing the affected quality characteristics
- Perform dynamic testing, including regression testing



Mitigate by testing

- Select experienced testers
- Use appropriate independence
- Conduct reviews and static analysis
- Apply suitable techniques and coverage



Other responses

- Risk acceptance
- Risk transfer
- Contingency planning



Monitor

- Check whether mitigation is effective
- Refine assessment
- Identify emerging risks



INTERACTIVE: Ask: when might a product risk be accepted rather than mitigated?

5.3 Test monitoring and test control

Monitoring gathers information; control takes corrective action.



Test monitoring

- concerned with **gathering information about testing**. This information is used to assess test progress and to measure whether the test exit criteria or the test tasks associated with the exit criteria are satisfied, such as meeting the targets for coverage of product risks, requirements, or acceptance criteria.



Test control

- uses the **information from test monitoring to provide, in a form of the control directives, guidance and the necessary corrective actions** to achieve the most effective and efficient testing. Examples of control directives include:
 - Reprioritizing tests when an identified risk becomes an issue
 - Re-evaluating whether a test item meets entry criteria or exit criteria due to rework
 - Adjusting the test schedule to address a delay in the delivery of the test environment
 - Adding new resources when and where needed



Test Completion

- collects data from completed test activities to consolidate experience, testware, and any other relevant information.**
- Test completion activities occur at project milestones such as when a test level is completed, an agile iteration is finished, a test project is completed (or cancelled), a software system is released, or a maintenance release is completed.



INTERACTIVE: Ask: what control action would you take if the test environment is delayed?

5.3.1 Metrics used in testing

Metrics support progress, quality, effectiveness and reporting.



Test metrics are gathered **to show progress against the planned schedule and budget**, the current quality of the test object, and the effectiveness of the test activities with respect to the objectives or an iteration goal. Test monitoring gathers a variety of metrics to support the test control and test completion.

Common test metrics include:



Progress

- task completion
- resource usage
- test effort
- cases run / not run / passed / failed



Quality and defects

- availability, response time, mean time to failure
- defect count, priorities, density, detection percentage



Risk / coverage / cost

- residual risk level
- requirements coverage and code coverage
- cost of testing and cost of quality

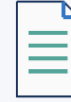
- Project progress metrics (e.g., task completion, resource usage, test effort)
- Test progress metrics (e.g., test case implementation progress, test environment preparation progress, number of test cases run/not run, passed/failed, test execution time)
- Product quality metrics (e.g., availability, response time, mean time to failure)
- Defect metrics (e.g., number and priorities of defects found/fixed, defect density, defect detection percentage)
- Risk metrics (e.g., residual risk level)
- Coverage metrics (e.g., requirements coverage, code coverage)
- Cost metrics (e.g., cost of testing, organizational cost of quality)



INTERACTIVE: Which metrics would you show to developers vs senior management?

5.3.2 Test reports

Reports communicate test information during and after testing.



Test reporting summarizes and communicates test information during and after testing.



Test progress report

- Ongoing control of testing
- Usually daily, weekly or periodic
- Includes progress, deviations, impediments, metrics, risks and next period plan

During test monitoring and control, the test team generates test progress reports for stakeholders to keep them informed. Test progress reports are usually generated on a regular basis (e.g., daily, weekly, etc.) and include:

- Test period
- Test progress (e.g., ahead or behind schedule), including any notable deviations
- Impediments for testing, and their workarounds
- Test metrics (see section 5.3.1 for examples)
- New and changed risks within testing period
- Testing planned for the next period



INTERACTIVE: Ask learners what information should go into a daily dashboard vs final report.

5.3.2 Test reports

Reports communicate test information during and after testing.

5



Test completion report

- Summarizes a completed stage
- Evaluates testing and product quality against plan and exit criteria
- Includes deviations, remaining risks, unfixed defects and lessons learned

Typical test completion reports include:

- Test summary
- Testing and product quality evaluation based on the original test plan (i.e., test objectives and exit criteria)
- Deviations from the test plan (e.g., differences from the planned schedule, duration, and effort).
- Testing impediments and workarounds
- Test metrics based on test progress reports
- Unmitigated risks, defects not fixed
- Lessons learned that are relevant to the testing

Different audiences require different information in the reports, and influence the degree of formality and the frequency of reporting. Reporting on test progress to others in the same team is often frequent and informal, while reporting on testing for a completed project follows a set template and occurs only once.



INTERACTIVE: Ask learners what information should go into a daily dashboard vs final report.



5.3.3 Communicating the status of testing

Choose communication channels based on audience and context.

The best means of communicating test status varies, depending on test management concerns, organizational test strategies, regulatory standards, or, in the case of self-organizing teams (see section 1.5.2), on the team itself.

The options include:



Informal / immediate

- verbal updates
- team chat
- task boards



Visual / ongoing

- dashboards
- CI/CD boards
- burn-down charts



Formal / distributed

- online documentation
- email
- formal reports for audits or governance

One or more of these options can be used. More formal communication may be more appropriate for distributed teams where direct face-to-face communication is not always possible due to geographical distance or time differences.

Typically, different stakeholders are interested in different types of information, so communication should be tailored accordingly.



INTERACTIVE: Ask: what would you communicate differently to testers, product owner and steering committee?

5.3.3 Communicating the status of testing

Choose communication channels based on audience and context.



Figure 5.15 shows an example of a typical dashboard for communicating information about the continuous integration and continuous delivery process. From this dashboard, you can very quickly get the most important basic information about the state of the process, for example:

- The number of releases by release status (successful, not built, aborted, etc.)
- The dates and status of the last ten releases
- Information about the last ten commits
- Frequency of releases
- Average time to create a release as a function of time



Fig. 5.15 Example of a CI/CD dashboard (source: <https://devops.com/electric-cloud-extends-continuous-delivery-platform/>)



INTERACTIVE: Ask: what would you communicate differently to testers, product owner and steering committee?

5.4 Configuration Management

CM keeps testing work products identifiable, controlled and reproducible.



Configuration items

- Provides a discipline for **identifying, controlling, and tracking work products** such as test plans, test strategies, test conditions, test cases, test scripts, test results, test logs, and test reports as configuration items.
- For a **complex configuration item** (e.g., a test environment), CM records the items it consists of, their relationships, and versions. If the configuration item is approved for testing, it becomes a baseline and can only be changed through a formal change control process.
- Configuration management keeps a record of changed configuration items when a new baseline is created. It is possible to revert to a previous baseline to reproduce previous test results.



Baseline

- Approved configuration item
- Changes only through formal control
- Supports reproducibility of previous results



Traceability

- Unique identification and version control
- Relationships between items are maintained
- Documentation references are unambiguous



INTERACTIVE: Ask: why must test results be tied to exact versions of software and testware?

5.4 Configuration Management

CM keeps testing work products identifiable, controlled and reproducible.

To properly support testing, configuration management may involve ensuring the following:

- All configuration items, including test items (individual parts of the test object), are uniquely identified, version controlled, tracked for changes, and related to other configuration items so that traceability can be maintained throughout the test process
- All identified documentation and software items are referenced unambiguously in test documentation

Continuous integration, continuous delivery, continuous deployment and the associated testing are typically implemented as part of an automated DevOps pipeline (see section 2.1.4), in which automated CM is normally included

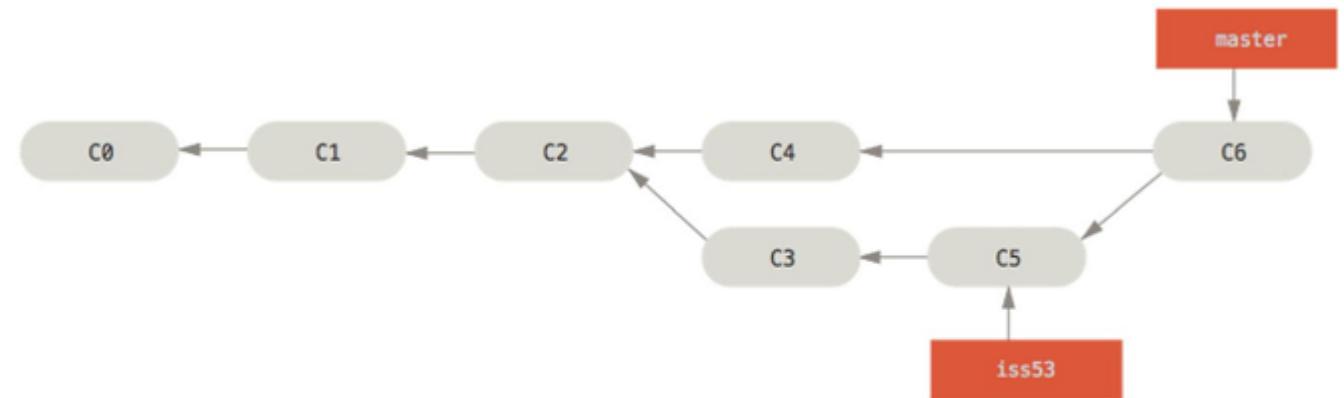


Figure 5.16 shows a graphical representation of a certain repository stored in the git code versioning system. The vertices C0, C1, ..., C6 denote so-called snapshots, i.e., successive versions of the source code. The arrows indicate on the basis of which version the next version was created.

Fig. 5.16 Graphical representation of a code repository in the Git system (source: <https://git-scm.com/docs/gittutorial>)



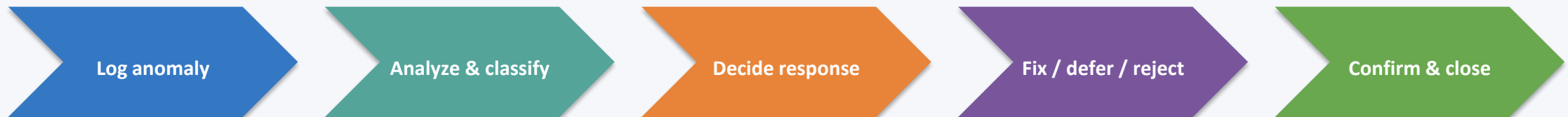
INTERACTIVE: Ask: why must test results be tied to exact versions of software and testware?



5.5 Defect management workflow

Defect management handles anomalies from discovery to closure.

- Since one of the **major test objectives is to find defects**, an established defect management process is essential. Although we refer to "defects" here, the reported anomalies may turn out to be real defects or something else (e.g., false positive, change request) - this is resolved during the process of dealing with the defect reports. Anomalies may be reported during any phase of the SDLC and the form depends on the SDLC.
- At a minimum, the defect management process includes a workflow for handling individual anomalies from their discovery to their closure and rules for their classification. The workflow typically comprises activities to log the reported anomalies, analyze and classify them, decide on a suitable response such as to fix or keep it as it is and finally to close the defect report. The process must be followed by all involved stakeholders. It is advisable to handle defects from static testing (especially static analysis) in a similar way.



Why it matters

- Provides information to resolve issues
- Tracks quality of work product
- Feeds improvement of development and test process



Important nuance

- Reported anomalies may be real defects, false positives or change requests
- The process resolves classification



INTERACTIVE: Ask: what information helps a developer reproduce a failure quickly?



5.5 Defect report contents

A good defect report helps reproduction, resolution and tracking.



Typical contents

- Unique identifier
- Title with a short summary of the anomaly being reported
- Date when the anomaly was observed, issuing organization, and author, including their role
- Identification of the test object and test environment
- Context of the defect (e.g., test case being run, test activity being performed, SDLC phase, and other relevant information such as the test technique, checklist or test data being used)
- Description of the failure to enable reproduction and resolution including the steps that detected the anomaly, and any relevant test logs, database dumps, screenshots, or recordings
- Expected results and actual results
- Severity of the defect (degree of impact) on the interests of stakeholders or requirements
- Priority to fix
- Status of the defect (e.g., open, deferred, duplicate, waiting to be fixed, awaiting confirmation testing, re-opened, closed, rejected)
- References (e.g., to the test case)



Typical defect reports:

- Provide developers and other parties with information about any adverse event that occurred, to enable them to identify specific effects, to isolate the problem with a minimal reproducing test, and to correct the potential defect(s), as needed or to otherwise resolve the problem
- Provide test managers a means of tracking the quality of the work product and the impact on the testing (e.g., if a lot of defects are reported, the testers will have spent a lot of time reporting them instead of running tests, and there will be more confirmation testing needed)
- Provide ideas for development and test process improvement



INTERACTIVE: Ask learners to rewrite a vague defect title into a useful one.



Any Question?



INTERACTIVE: Run a mini-quiz: calculate a three-point estimate and prioritize three tests.